

Distributed Systems

DAT520 - Spring 2026

Chapter 4 - Shared Memory

Prof. Hein Meling

Motivation for Shared Memory

Why Shared Memory?

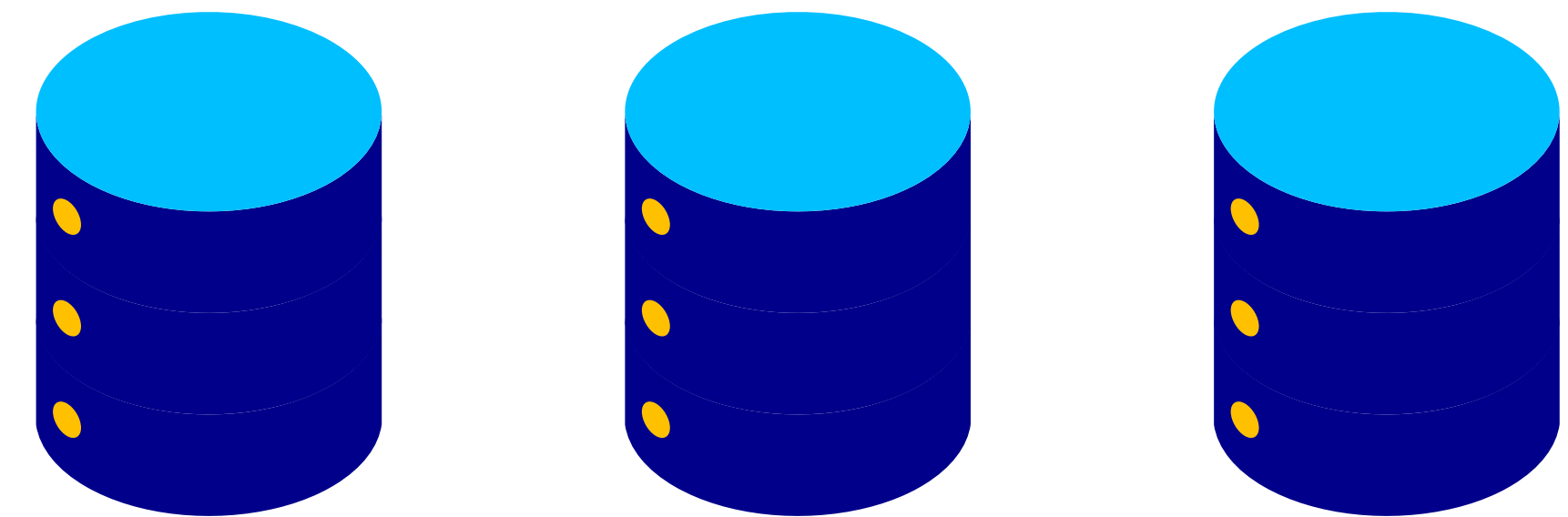
- Distributed processes communicate using **message passing**
- However, reasoning about message exchanges can be **complex**
- A **shared memory abstraction** provides a simpler programming model

Shared Memory: Idea

- Processes interact through **read/write operations** on shared variables
- Conceptually similar to **hardware registers** in multiprocessor systems
- Benefit
 - Programmers can reason about **data instead of messages**

Shared Memory: Examples of Systems

- Distributed file systems
- Shared collaborative workspaces
- Distributed (and replicated) storage
- Coordination primitives



The Register Abstraction

A ***distributed programming abstraction*** encapsulating read-write storage shared among processes

Key idea

- Processes interact through **read** and **write** operations

The Register Abstraction

Analogy: Similar to **CPU registers** in shared-memory multiprocessors

Important difference

- In distributed systems:
 - No physical shared memory
 - Processes communicate through messages

Thus registers must be **implemented using message passing**

The Register Abstraction

Registers provide a **conceptual layer** above message passing.

They simplify reasoning about:

- Distributed storage
- Consistency of replicated data
- Coordination between processes

Types of Registers

- Three main consistency guarantees:
 - (Safe registers)
 - Regular registers
 - Atomic registers
- These differ in how **reads behave when concurrent with writes**

Types of Registers

Atomic registers provide the **strongest consistency**

Safe < Regular < Atomic

Register Notation

Registers are classified by **number of writers and readers**

Notation: **(X, Y)**

- **X** = number of writers
- **Y** = number of readers

Register Notation: Examples

$(1,N)$ register

- One designated writer
- Any process may read

(N,N) register

- Any process may read and write

Register Operations

Registers support two operations:

Write(v)

- Stores value v in the register

Read()

- Returns the current value stored

Assumptions for simplicity:

- Register values are **integers**
- Initial value is **0**

(may store any data type in practice)

Value Identification

Every **written value** is uniquely identified by:

⟨ process ID, timestamp, value ⟩

Why?

- Distinguish **multiple writes**
- Resolve **concurrent updates**
- Determine **which value is the latest**

This metadata is essential for implementing registers over message passing

Correctness Properties

Liveness

Every operation eventually completes

- No read or write blocks forever

Safety

Every read returns the ***last value written*** (LVW)

I need to write!!

Let me read it!

No, it's my turn!

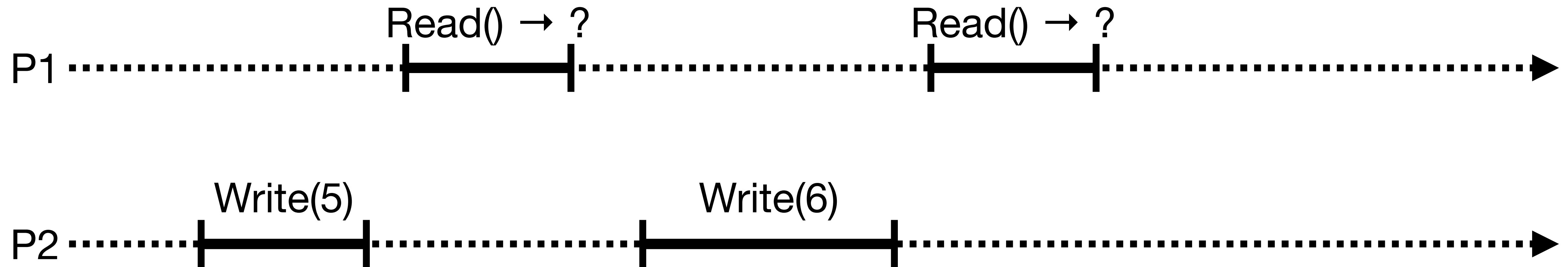
REGISTER

<?, ?, ?>

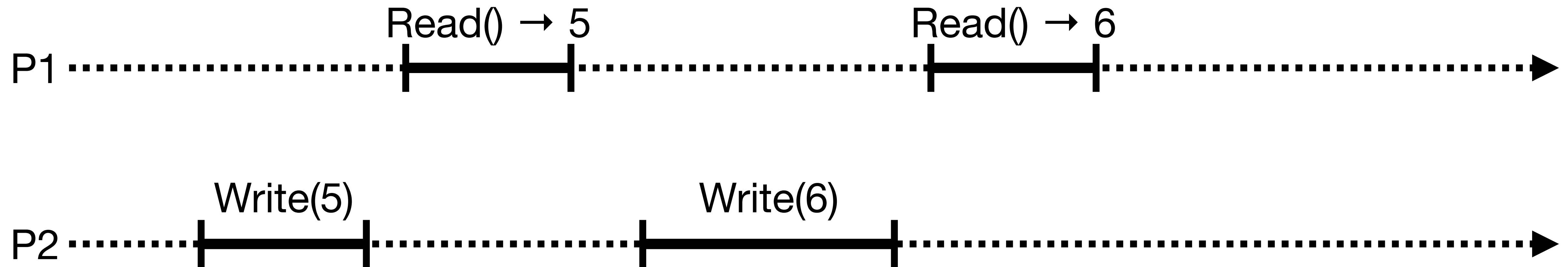


Example Scenarios

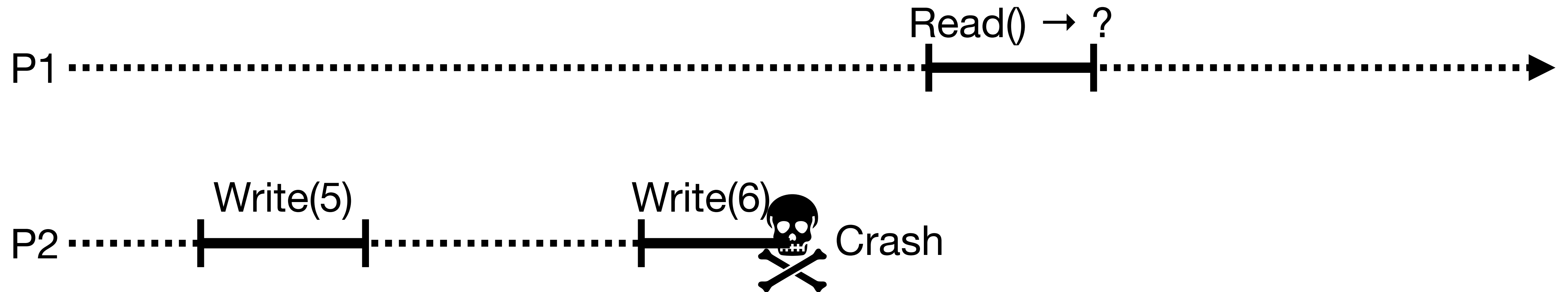
Sequential Execution: Read Returns Last Value Written



Sequential Execution: Read Returns Last Value Written



Execution with Failures



Execution with Failures: Two Cases

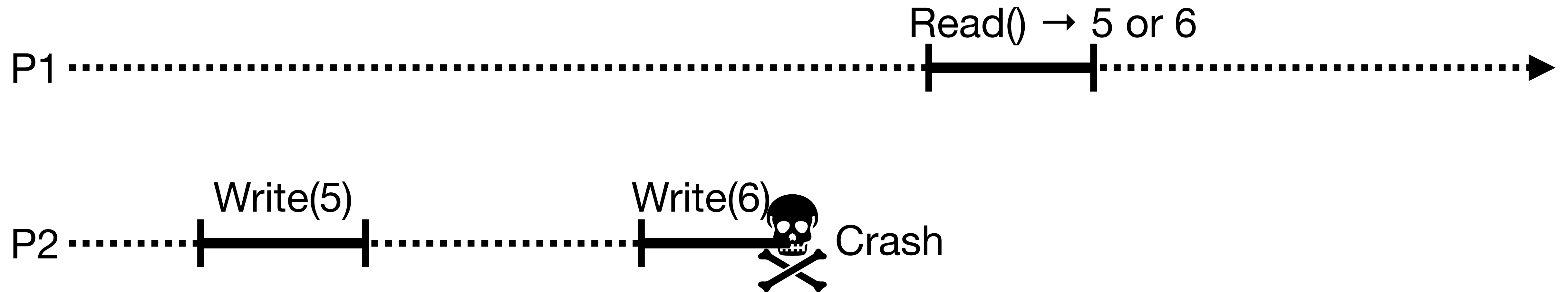
1. No future read should return the value of a failed write

Behave as if write **never occurred**

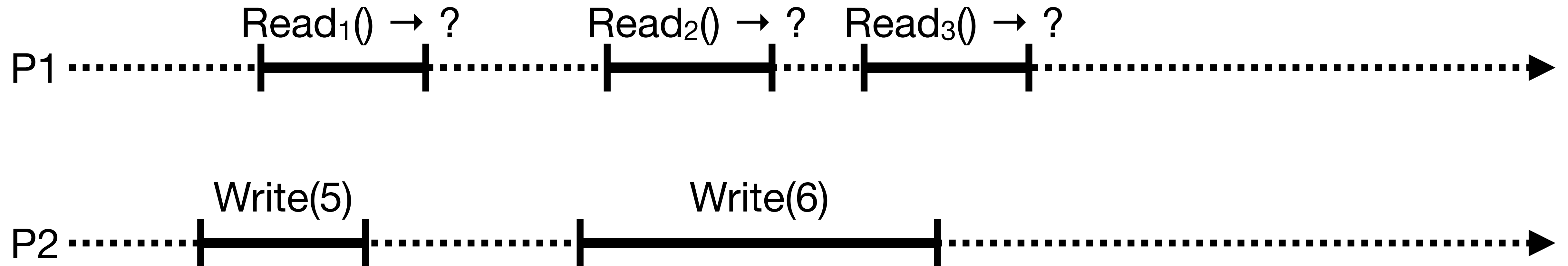
2. Return value of last (failing) write

Behave as if write **actually completed**

Execution with Failures: Both Valid



Concurrent Execution



Concurrent Execution: Questions

1. What value should a **read** return if it is **concurrent with some write**?

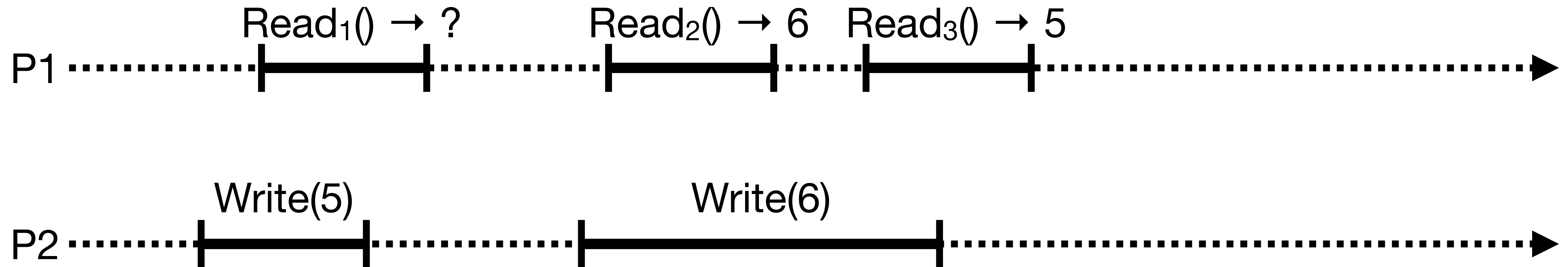
In this situation, what does “**last value written**” actually mean?

2. If **two writes occur concurrently**:

What is the **last value written**?

3. Can two consecutive reads return different values?

Concurrent Execution: Is this allowed?



Can Read₂() return 6, and a later Read₃() return 5?

These are the Questions we
will Answer in this Chapter!

Regular and Atomic registers differ in how they handle Concurrent Operations

Operation Completion

An operation ***op*** invoked by process ***p*** is **complete** if:

- Process ***p*** does not crash
- Before receiving confirmation

Confirmation means:

- Read: process receives the returned value
 - Write: process receives a confirmation message
- ✓ The operation finished successfully

Failed Operations

An operation is **failed** if:

- It was invoked by process ***p***
- But ***p*** crashes before receiving confirmation

Precedence of Operations

We say that

$op_A @ p$ precedes $op_B @ q$

If

op_A is **complete**, and

op_B **starts after** op_A finishes

Sequential vs Concurrent Operations

Two operations are **sequential** if:

- One **precedes** the other

Two operations are **concurrent** if:

- **Neither operation precedes the other**
- The operations **overlap in time**

$(1, N)$ Regular Register

Regular Register: Model

A (1,N) regular register has:

- One writer process ***p***
- N readers (any process may read)

Operations

- `reg.Write(v)` — invoked only by ***p***
- `reg.Read()` — invoked by any process

Goal

Provide **strong guarantees** when there is **no concurrency**

Provide **minimal guarantees** when **operations overlap or fail**

Regular Register: Semantics

Case 1 — No Concurrent Write

If Read() is **not concurrent** with any Write():

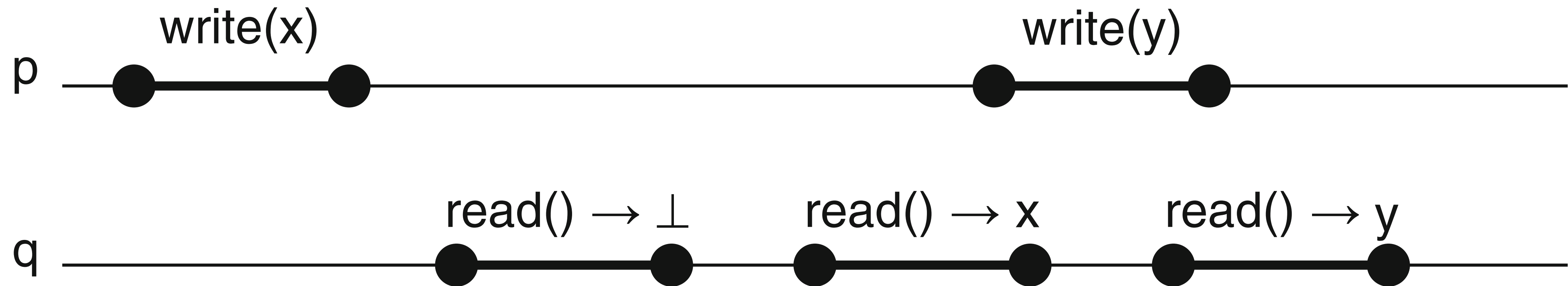
Read() → last value written

Case 2 — Concurrent Write

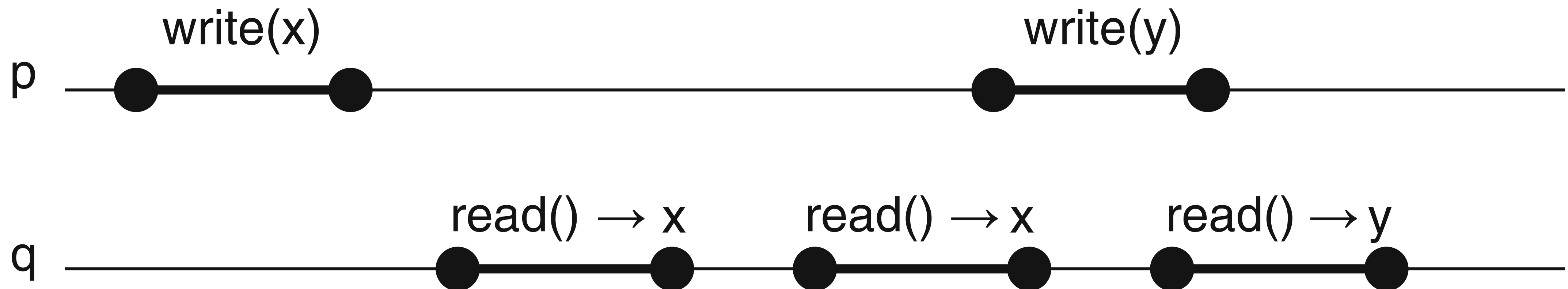
If Read() is **concurrent** with a Write():

Read() → $\begin{cases} \text{last value written} \\ \text{value currently being written} \end{cases}$

Regular Register: Invalid Execution – Non-Regular



Regular Register: Valid Execution – Regular



Module 4.1: Interface and properties of a $(1, N)$ regular register

Module:

Name: $(1, N)$ -RegularRegister, **instance** *onrr*.

Events:

Request: $\langle \textit{onrr}, \textit{Read} \rangle$: Invokes a read operation on the register.

Request: $\langle \textit{onrr}, \textit{Write} \mid v \rangle$: Invokes a write operation with value v on the register.

Indication: $\langle \textit{onrr}, \textit{ReadReturn} \mid v \rangle$: Completes a read operation on the register with return value v .

Indication: $\langle \textit{onrr}, \textit{WriteReturn} \rangle$: Completes a write operation on the register.

Properties:

ONRR1: Termination: If a correct process invokes an operation, then the operation eventually completes.

ONRR2: Validity: A read that is not concurrent with a write returns the last value written; a read that is concurrent with a write returns the last value written or the value concurrently written.

Regular Register: ROWA Algorithm

Idea: Each process stores a **local copy** of the register value

Operations:

- **Write:** propagate new value to **all** processes
- **Read:** return the **local value**

Regular Register: ROWA Algorithm

Write

1. Broadcast the new value
2. All processes update their local register
3. All processes send ACK

Read

Purely local; *no network communication required*

Algorithm 4.1: Read-One Write-All

Implements:

$(1, N)$ -RegularRegister, **instance** *onrr*.

Uses:

BestEffortBroadcast, **instance** *beb*;
 PerfectPointToPointLinks, **instance** *pl*;
 PerfectFailureDetector, **instance** $\mathcal{P}$.

upon event $\langle \textit{onrr}, \textit{Init} \rangle$ **do**

$\textit{val} := \perp$;
 $\textit{correct} := \Pi$;
 $\textit{writeset} := \emptyset$;

upon event $\langle \mathcal{P}, \textit{Crash} \mid p \rangle$ **do**

$\textit{correct} := \textit{correct} \setminus \{p\}$;

upon event $\langle \textit{onrr}, \textit{Read} \rangle$ **do**

trigger $\langle \textit{onrr}, \textit{ReadReturn} \mid \textit{val} \rangle$;

upon event $\langle \textit{onrr}, \textit{Write} \mid v \rangle$ **do**

trigger $\langle \textit{beb}, \textit{Broadcast} \mid [\text{WRITE}, v] \rangle$;

upon event $\langle \textit{beb}, \textit{Deliver} \mid q, [\text{WRITE}, v] \rangle$ **do**

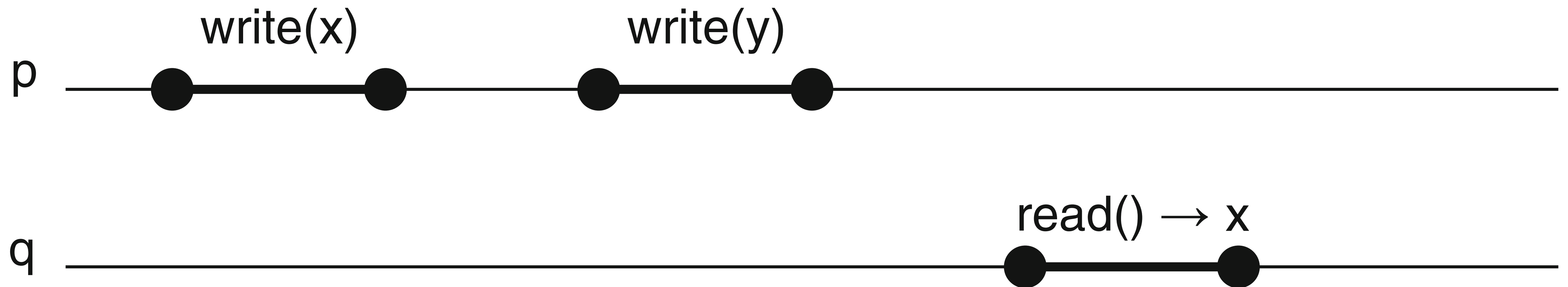
$\textit{val} := v$;
trigger $\langle \textit{pl}, \textit{Send} \mid q, \text{ACK} \rangle$;

upon event $\langle \textit{pl}, \textit{Deliver} \mid p, \text{ACK} \rangle$ **do**

$\textit{writeset} := \textit{writeset} \cup \{p\}$;

upon $\textit{correct} \subseteq \textit{writeset}$ **do**

$\textit{writeset} := \emptyset$;
trigger $\langle \textit{onrr}, \textit{WriteReturn} \rangle$;



If the PFD's assumptions are violated!

Regular Register: Majority Voting Algorithm

Idea: Both reads and writes communicate with a **majority of processes**

(Use quorums instead of relying on perfect failure detector)

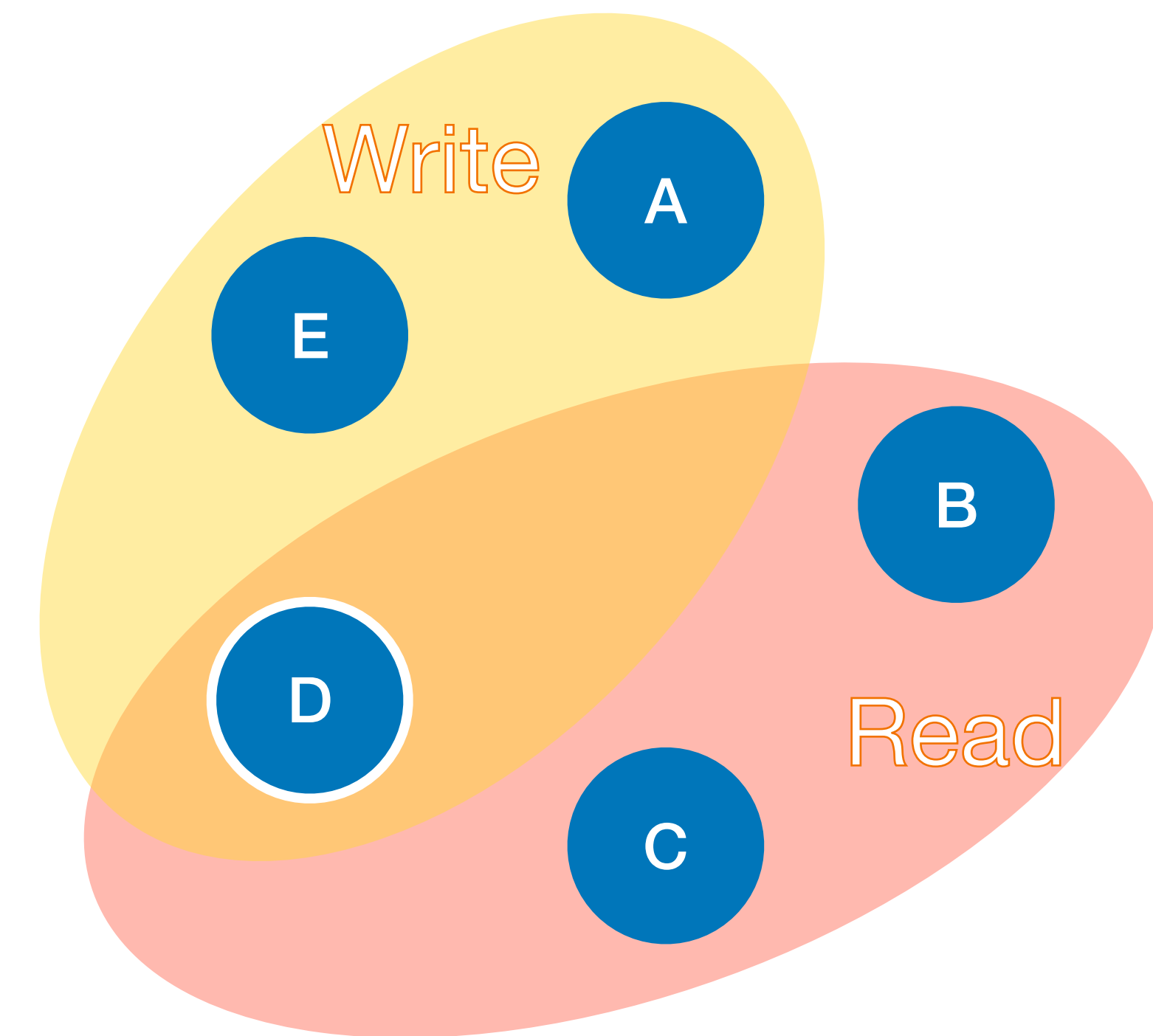
Witness Processes

*A set of processes act as **witnesses** for the register value*

Quorum Intersection

Property: *Any two majorities intersect*

At least **one witness participates in both operations**



Regular Register: Majority Voting Algorithm

Write

1. Send value to all processes
2. Wait for **ACKs from a majority**

Read

1. Query processes
2. Collect responses from a **majority**
3. Return the **most recent value**

Algorithm 4.2: Majority Voting Regular Register

Implements:

$(1, N)$ -RegularRegister, **instance** *onrr*.

Uses:

BestEffortBroadcast, **instance** *beb*;

PerfectPointToPointLinks, **instance** *pl*.

upon event $\langle \textit{onrr}, \textit{Init} \rangle$ **do**

$(ts, val) := (0, \perp);$

$wts := 0;$

$acks := 0;$

$rid := 0;$

$readlist := [\perp]^N;$

upon event $\langle \textit{onrr}, \textit{Write} \mid v \rangle$ **do**

$wts := wts + 1;$

$acks := 0;$

trigger $\langle \textit{beb}, \textit{Broadcast} \mid [\textit{WRITE}, wts, v] \rangle;$

upon event $\langle \textit{beb}, \textit{Deliver} \mid p, [\textit{WRITE}, ts', v'] \rangle$ **do**

if $ts' > ts$ **then**

$(ts, val) := (ts', v');$

trigger $\langle \textit{pl}, \textit{Send} \mid p, [\textit{ACK}, ts'] \rangle;$

upon event $\langle \textit{pl}, \textit{Deliver} \mid q, [\textit{ACK}, ts'] \rangle$ **such that** $ts' = wts$ **do**

$acks := acks + 1;$

if $acks > N/2$ **then**

$acks := 0;$

trigger $\langle \textit{onrr}, \textit{WriteReturn} \rangle;$

upon event $\langle \textit{onrr}, \textit{Read} \rangle$ **do**

$rid := rid + 1;$

$readlist := [\perp]^N;$

trigger $\langle \textit{beb}, \textit{Broadcast} \mid [\textit{READ}, rid] \rangle;$

upon event $\langle \textit{beb}, \textit{Deliver} \mid p, [\textit{READ}, r] \rangle$ **do**

trigger $\langle \textit{pl}, \textit{Send} \mid p, [\textit{VALUE}, r, ts, val] \rangle;$

upon event $\langle \textit{pl}, \textit{Deliver} \mid q, [\textit{VALUE}, r, ts', v'] \rangle$ **such that** $r = rid$ **do**

$readlist[q] := (ts', v');$

if $\#(readlist) > N/2$ **then**

$v := \textit{highestval}(readlist);$

$readlist := [\perp]^N;$

trigger $\langle \textit{onrr}, \textit{ReadReturn} \mid v \rangle;$

Regular Register: Summary

ROWA

- Reads are local
- Writes require all processes
- Requires perfect failure detector

Majority Voting

- Both reads and writes involve communication
- No failure detector required
- Uses quorum intersection



That's the problem with regular registers...

“..reading past values out of order!”

$(1, N)$ Atomic Register

Recall Regular Register: Semantics

Case 1 — No Concurrent Write

If Read() is **not concurrent** with any Write():

Read() → last value written

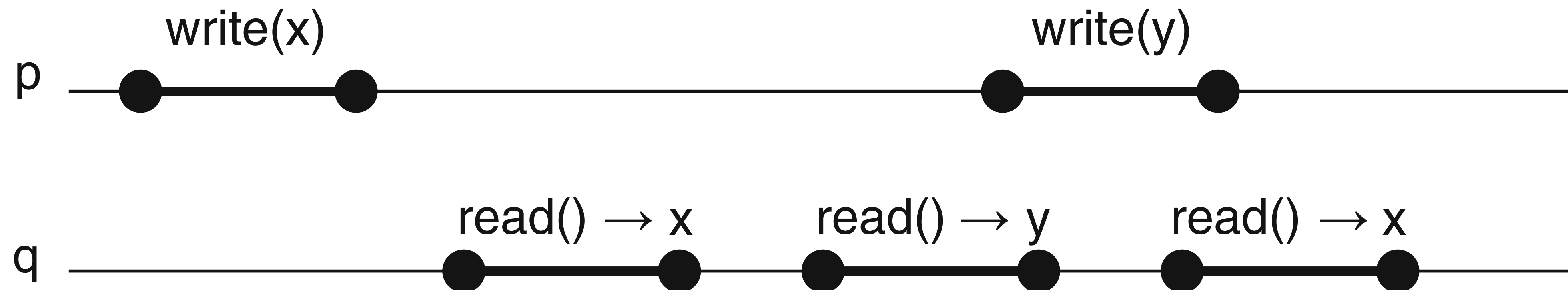
Case 2 — Concurrent Write

If Read() is **concurrent** with a Write():

Read() → $\begin{cases} \text{last value written} \\ \text{value currently being written} \end{cases}$

Recall Regular Register: Semantics

The **regular register specification** *allows* the following execution:



A **read** may return an **old value** even if a **new write is ongoing**

Reason: read || write, and the *write may not have completed updating the register*

This behavior may seem
strange in practice ...

Module 4.2: Interface and properties of a $(1, N)$ atomic register

Module:

Name: $(1, N)$ -AtomicRegister, **instance** *onar*.

Events:

Request: $\langle onar, Read \rangle$: Invokes a read operation on the register.

Request: $\langle onar, Write \mid v \rangle$: Invokes a write operation with value v on the register.

Indication: $\langle onar, ReadReturn \mid v \rangle$: Completes a read operation on the register with return value v .

Indication: $\langle onar, WriteReturn \rangle$: Completes a write operation on the register.

Properties:

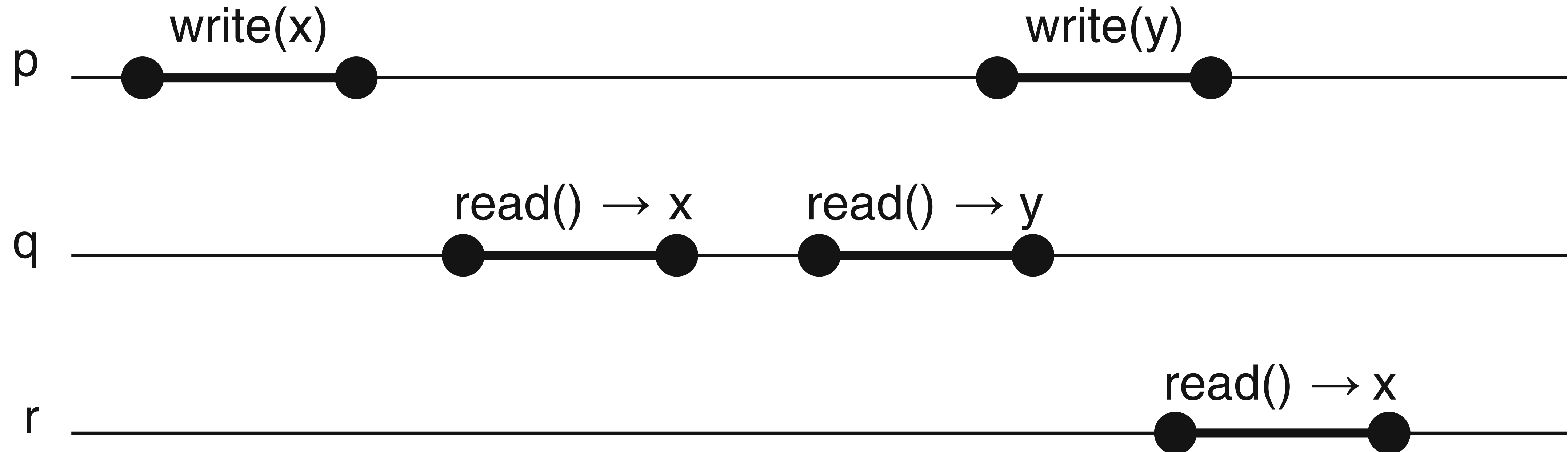
ONAR1–ONAR2: Same as properties ONRR1–ONRR2 of a $(1, N)$ regular register (Module 4.1).

ONAR3: Ordering: If a read returns a value v and a subsequent read returns a value w , then the write of w does not precede the write of v .

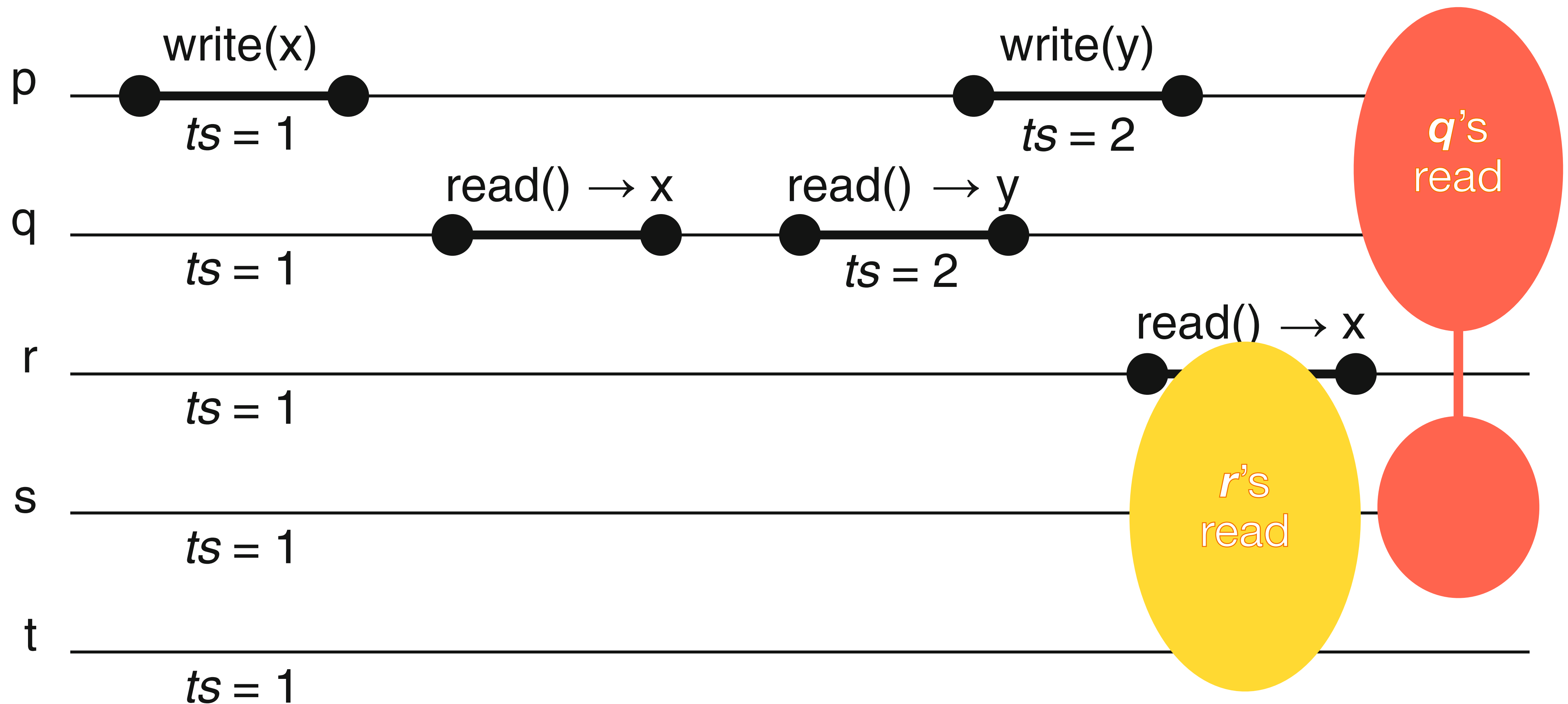
Counterexample: ROWA is not Atomic

Even though ***r***'s read occurs after ***q***'s read, it returns an older value

This **violates the ordering** we expect from atomic registers



Counterexample: Majority Voting is also not Atomic



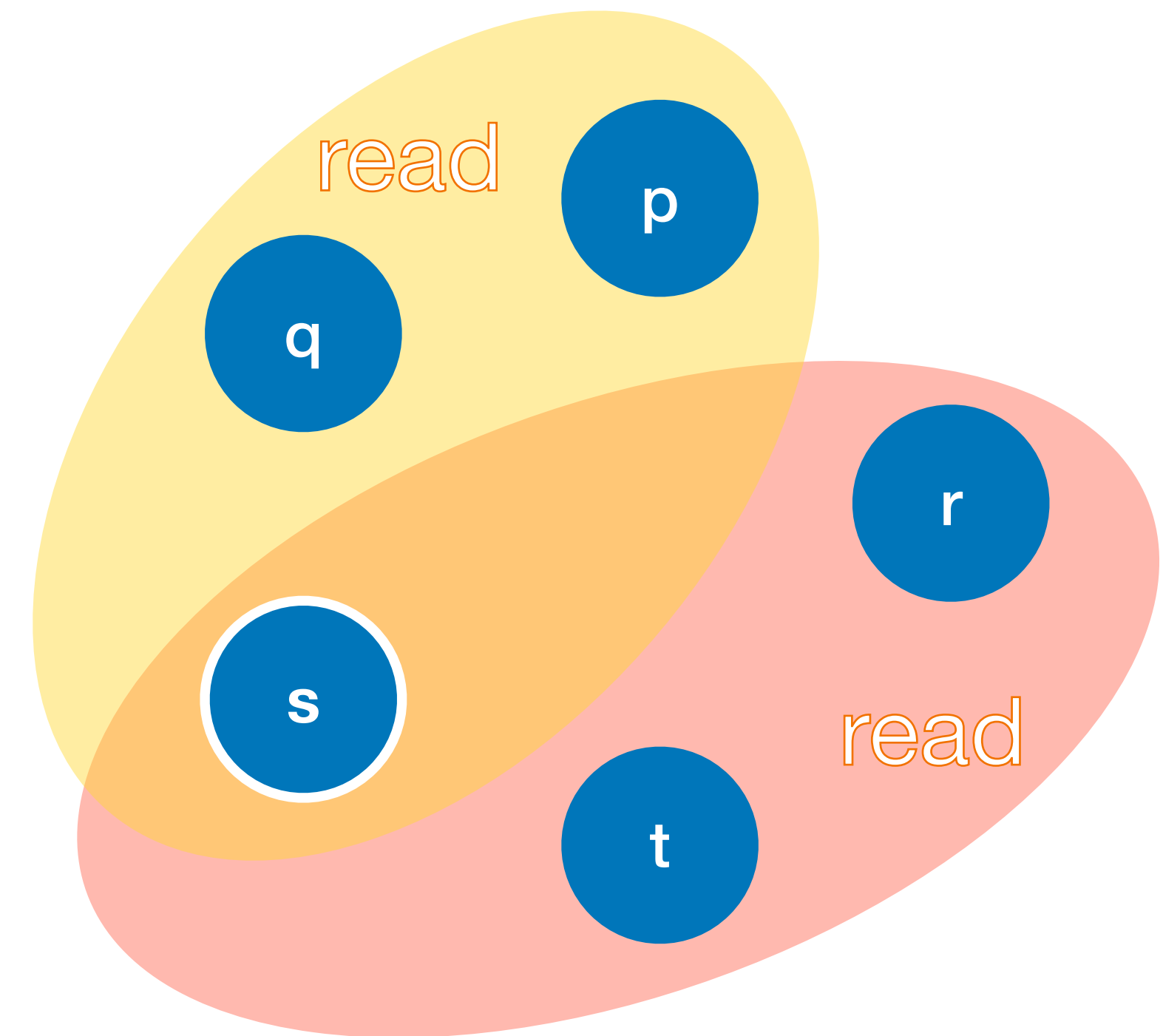
Counterexample: Majority Voting is also not Atomic

Even with quorum intersections, the algorithm can still produce executions where:

q's read quorum : {p, q, **s**}

r's read quorum: {r, **s**, t}

This **violates the ordering** we expect from atomic registers



Atomic Register
=
Regular Register
+
Ordering Property

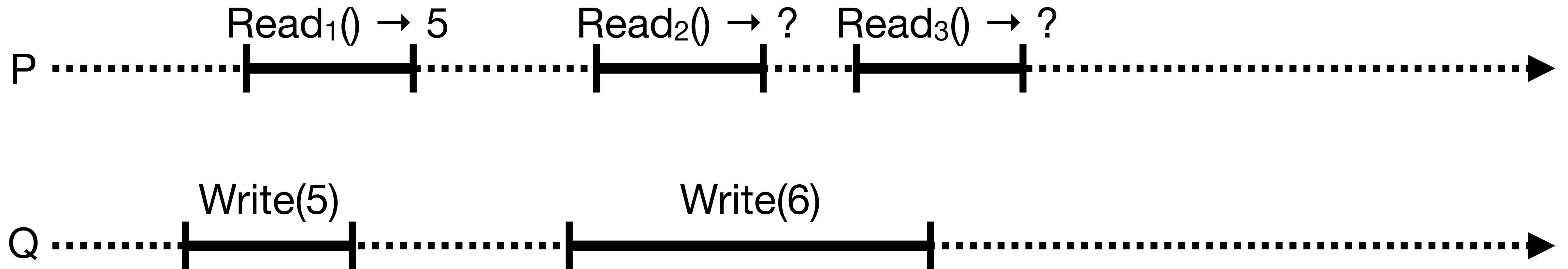
Ordering Property

Prevents an old value from being read once a new value has been read.

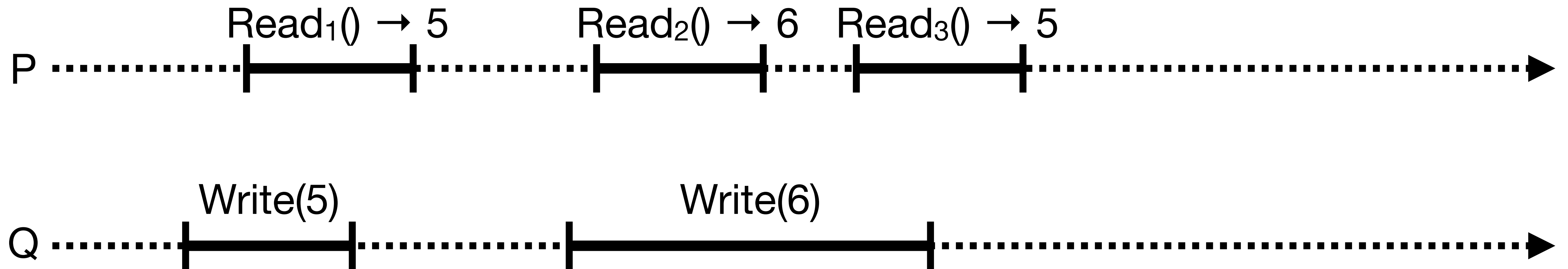
In other words:

Reads must observe writes in monotonic order.

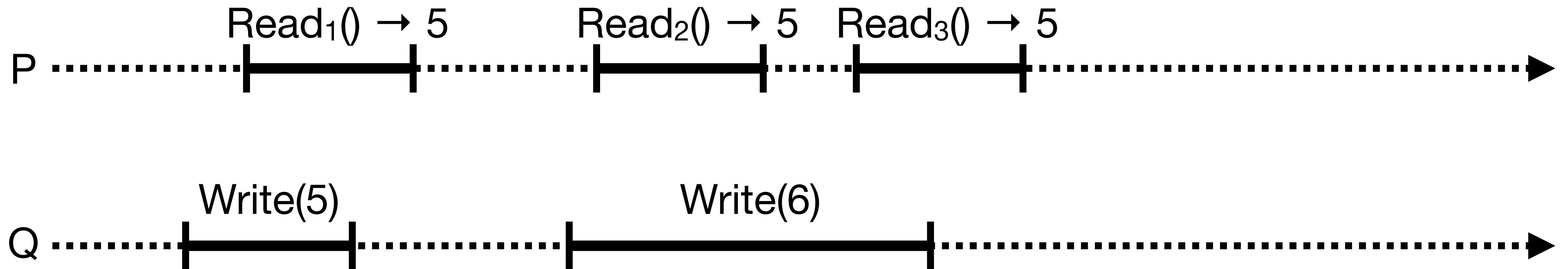
What are the Valid Atomic Executions?



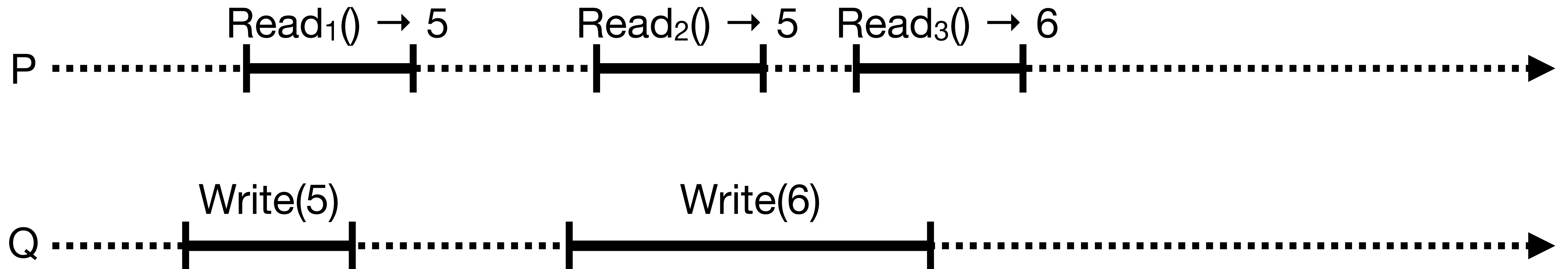
Atomic: Invalid Execution



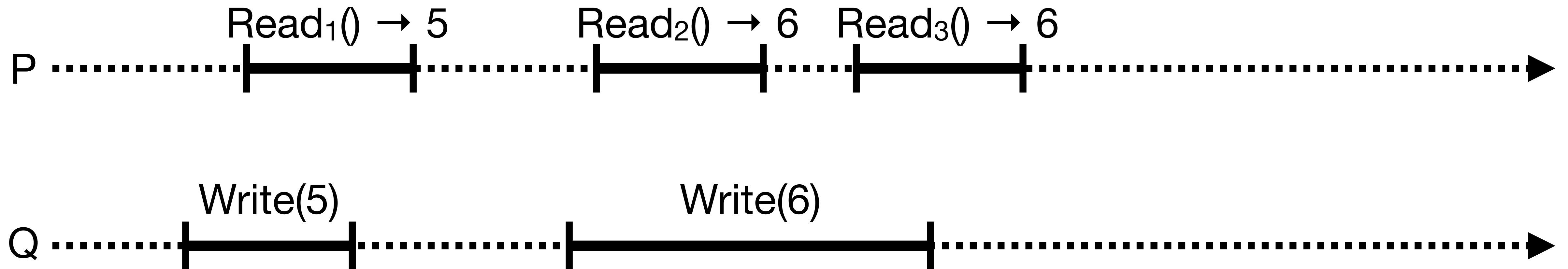
Atomic Execution



Atomic Execution



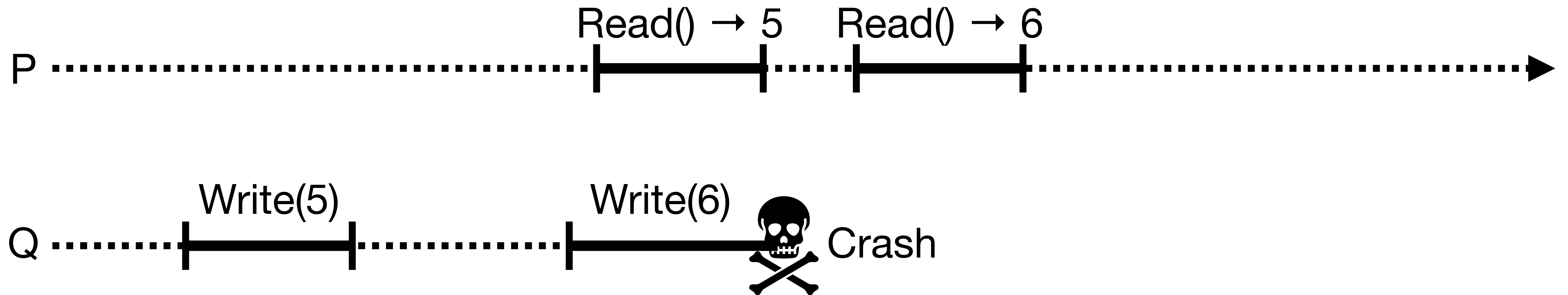
Atomic Execution



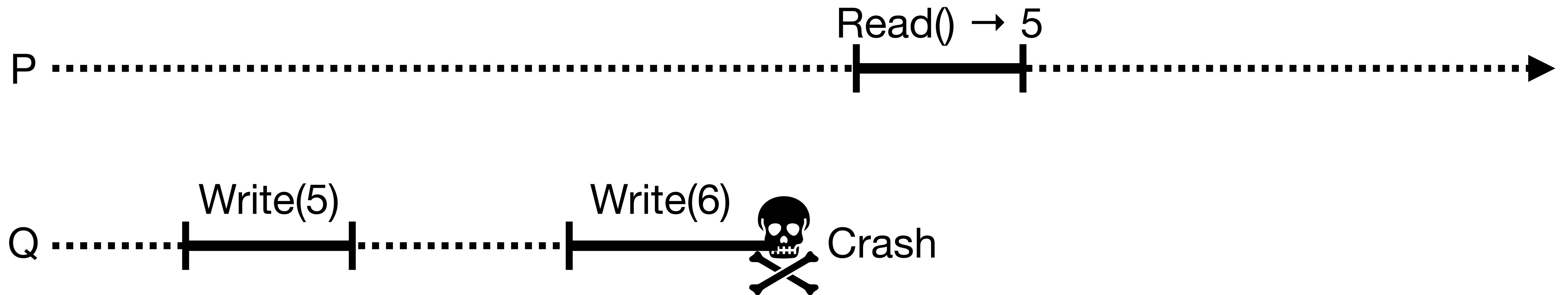
Atomic Execution: Example Summarized

Read ₂	Read ₃	
5	5	Valid
5	6	Valid
6	5	Invalid
6	6	Valid

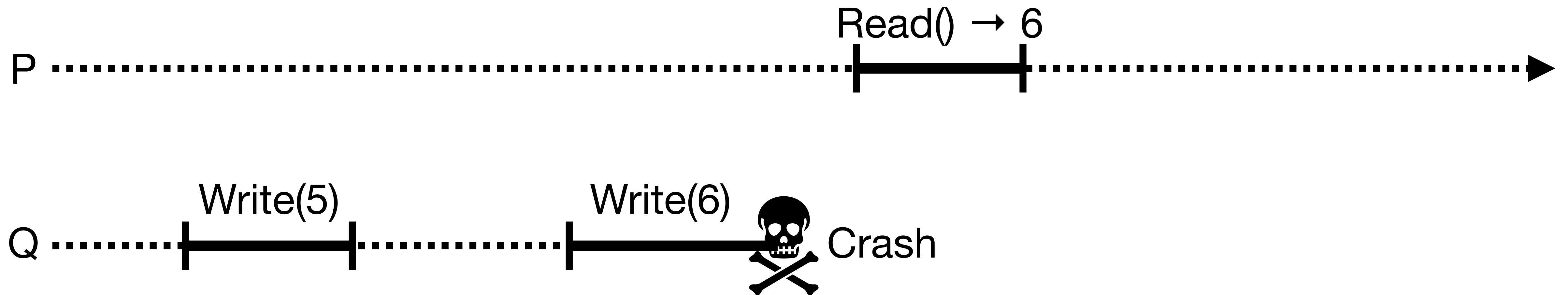
Atomic Execution with Crash



Atomic Execution with Crash



Atomic Execution with Crash



Atomic Registers

An **atomic register** provides **strong consistency guarantees**

Properties hold even when:

- Operations execute concurrently
- Processes fail

Atomic Register: Transformations (Ch 4.3.2 & 4.4.3)

$(1,N)$ Regular $\rightarrow$ $(1,1)$ Atomic

$(1,1)$ Atomic $\rightarrow$ $(1,N)$ Atomic

$(1,N)$ Atomic $\rightarrow$ (N,N) Atomic

Algorithm 4.5 Read-Impose Write-All (RIWA)

- Idea: Maintain a timestamped-value pair

On Read():

1. Read local value
2. Write-back this value (same timestamp)

Purpose: Ensure that **other processes observe latest value**

Prevents **future reads** from returning older values

Algorithm 4.5 Read-Impose Write-All (RIWA)

- Idea: Maintain a timestamped-value pair

On Write(val):

1. Increase timestamp ts
2. Broadcast (ts, val)
3. Receive (ts, val) : Update local value only if $ts > \text{local } ts$

Algorithm 4.5 Read-Impose Write-All (RIWA)

Performance:

- Two communication steps (both read and write)
- At most $O(N)$ messages
- Both operations require
 - Broadcast
 - Acknowledgments

Ch 4.3 (1,N) Atomic Register

Algorithm 4.5: Read-Impose Write-All

Implements:

$(1, N)$ -AtomicRegister, **instance** *onar*.

Uses:

BestEffortBroadcast, **instance** *beb*;

PerfectPointToPointLinks, **instance** *pl*;

PerfectFailureDetector, **instance** $\mathcal{P}$.

upon event $\langle onar, Init \rangle$ **do**

$(ts, val) := (0, \perp)$;

correct $:= \Pi$;

writeset $:= \emptyset$;

readval $:= \perp$;

reading $:= \text{FALSE}$;

upon event $\langle \mathcal{P}, Crash \mid p \rangle$ **do**

correct $:= correct \setminus \{p\}$;

upon event $\langle onar, Read \rangle$ **do**

reading $:= \text{TRUE}$;

readval $:= val$;

trigger $\langle beb, Broadcast \mid [WRITE, ts, val] \rangle$;

upon event $\langle onar, Write \mid v \rangle$ **do**

trigger $\langle beb, Broadcast \mid [WRITE, ts + 1, v] \rangle$;

upon event $\langle beb, Deliver \mid p, [WRITE, ts', v'] \rangle$ **do**

if $ts' > ts$ **then**

$(ts, val) := (ts', v')$;

trigger $\langle pl, Send \mid p, [ACK] \rangle$;

upon event $\langle pl, Deliver \mid p, [ACK] \rangle$ **then**

writeset $:= writeset \cup \{p\}$;

upon $correct \subseteq writeset$ **do**

writeset $:= \emptyset$;

if *reading* $= \text{TRUE}$ **then**

reading $:= \text{FALSE}$;

trigger $\langle onar, ReadReturn \mid readval \rangle$;

else

trigger $\langle onar, WriteReturn \rangle$;

Algorithm 4.6/4.7 Read-Impose Write-Majority (RIWM)

- Idea: Maintain a timestamped-value pair with majorities

On Read():

1. Query a majority and collect (ts, val)-pairs
2. Choose (ts, val)-pair with largest timestamp
3. Broadcast this value to a majority

Purpose: Ensure that the chosen value is **widely adopted**

Algorithm 4.6/4.7 Read-Impose Write-Majority (RIWM)

- Idea: Maintain a timestamped-value pair with majorities

On Write():

1. Broadcast (ts, val) to all
2. Wait for **ACKs from a majority**

Purpose: Ensure that a **majority adopts the new value**

Algorithm 4.5 Read-Impose Write-Majority (RIWM)

Performance:

- Two communication steps: Write ($2N$ messages)
- Four communication steps: Read ($4N$ messages)
- At most $O(N)$ messages

Algorithm 4.6: Read-Impose Write-Majority (part 1, read)

Implements:
 $(1, N)$ -AtomicRegister, **instance** *onar*.

Uses:

 BestEffortBroadcast, **instance** *beb*;

 PerfectPointToPointLinks, **instance** *pl*.

upon event $\langle onar, Init \rangle$ **do**
 $(ts, val) := (0, \perp);$
 $wts := 0;$
 $acks := 0;$
 $rid := 0;$
 $readlist := [\perp]^N;$
 $readval := \perp;$
 $reading := \text{FALSE};$
upon event $\langle onar, Read \rangle$ **do**
 $rid := rid + 1;$
 $acks := 0;$
 $readlist := [\perp]^N;$
 $reading := \text{TRUE};$
trigger $\langle beb, Broadcast \mid [\text{READ}, rid] \rangle;$
upon event $\langle beb, Deliver \mid p, [\text{READ}, r] \rangle$ **do**
trigger $\langle pl, Send \mid p, [\text{VALUE}, r, ts, val] \rangle;$
upon event $\langle pl, Deliver \mid q, [\text{VALUE}, r, ts', v'] \rangle$ **such that** $r = rid$ **do**
 $readlist[q] := (ts', v');$
if $\#(readlist) > N/2$ **then**
 $(maxts, readval) := \text{highest}(readlist);$
 $readlist := [\perp]^N;$
trigger $\langle beb, Broadcast \mid [\text{WRITE}, rid, maxts, readval] \rangle;$
upon event $\langle onar, Write \mid v \rangle$ **do**
 $rid := rid + 1;$
 $wts := wts + 1;$
 $acks := 0;$
trigger $\langle beb, Broadcast \mid [\text{WRITE}, rid, wts, v] \rangle;$
upon event $\langle beb, Deliver \mid p, [\text{WRITE}, r, ts', v'] \rangle$ **do**
if $ts' > ts$ **then**
 $(ts, val) := (ts', v');$
trigger $\langle pl, Send \mid p, [\text{ACK}, r] \rangle;$
upon event $\langle pl, Deliver \mid q, [\text{ACK}, r] \rangle$ **such that** $r = rid$ **do**
 $acks := acks + 1;$
if $acks > N/2$ **then**
 $acks := 0;$
if $reading = \text{TRUE}$ **then**
 $reading := \text{FALSE};$
trigger $\langle onar, ReadReturn \mid readval \rangle;$
else
trigger $\langle onar, WriteReturn \rangle;$

Atomic Registers: Key Takeaways

Atomic registers enforce a stronger guarantee:

Reads cannot observe values out of order

Techniques used:

- **timestamps**
- **write-back**
- **quorum intersections**

(N,N) Atomic Register

Multiple Readers and Writers

An (N,N) atomic register allows:

- Any process to **read** and **write**

Goal: Maintain **atomic semantics** despite:

- Concurrent writes
- Concurrent reads
- Failures

Multiple Writers: Challenges

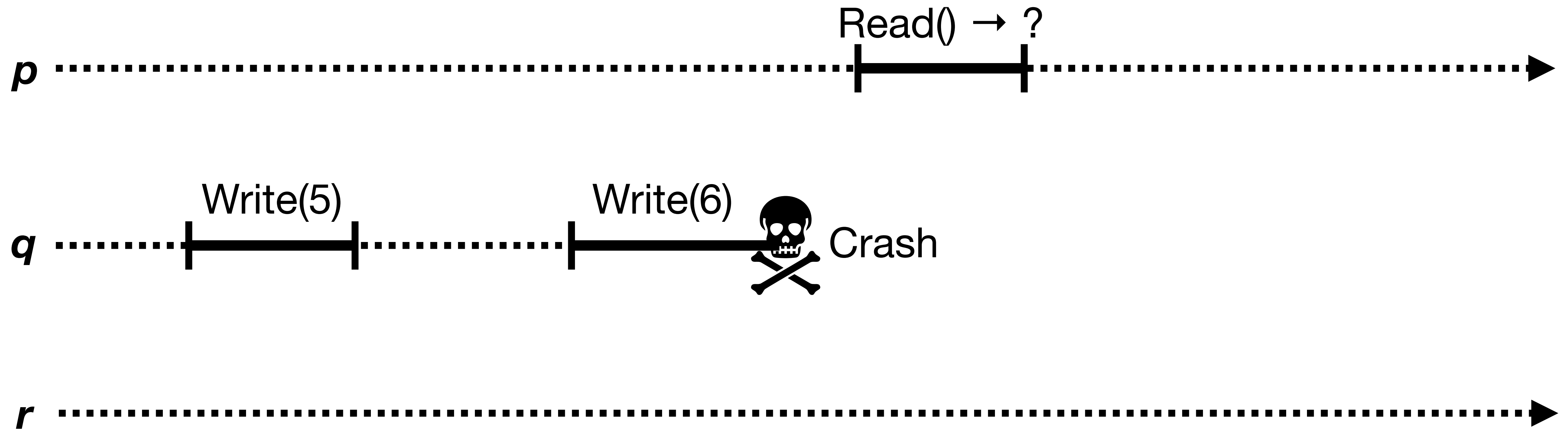
Consider two processes writing concurrently:

- p writes v
- q writes w

Questions

- What value should a **read** return?
- What about **multiple concurrent reads**?
- What happens if a **write fails**?

Atomic Execution with Crash: What Should the Read Return?



Module 4.3: Interface and properties of an (N, N) atomic register

Module:

Name: (N, N) -AtomicRegister, **instance** $nnar$.

Events:

Request: $\langle nnar, Read \rangle$: Invokes a read operation on the register.

Request: $\langle nnar, Write \mid v \rangle$: Invokes a write operation with value v on the register.

Indication: $\langle nnar, ReadReturn \mid v \rangle$: Completes a read operation on the register with return value v .

Indication: $\langle nnar, WriteReturn \rangle$: Completes a write operation on the register.

Properties:

NNAR1: Termination: Same as property ONAR1 of a $(1, N)$ atomic register (Module 4.2).

NNAR2: Atomicity: Every read operation returns the value that was written most recently in a hypothetical execution, where every failed operation appears to be complete or does not appear to have been invoked at all, and every complete operation appears to have been executed at some instant between its invocation and its completion.

Multiple Writers: Failed Writes

A failed write must appear as either:

- **Completed** or
- **Never invoked**

Readers must not observe **partial writes**

NNAR2: Atomicity: Every read operation returns the value that was written most recently in a hypothetical execution, where every failed operation appears to be complete or does not appear to have been invoked at all, and every complete operation appears to have been executed at some instant between its invocation and its completion.

Multiple Writers: Single Instant

Each operation appears to occur at a **single instant** (linearization point)

The execution must appear

Invocation●..... Confirmation

- **Sequential**
- **Failure-free**

NNAR2: Atomicity: Every read operation returns the value that was written most recently in a hypothetical execution, where every failed operation appears to be complete or does not appear to have been invoked at all, and every complete operation appears to have been executed at some instant between its invocation and its completion.

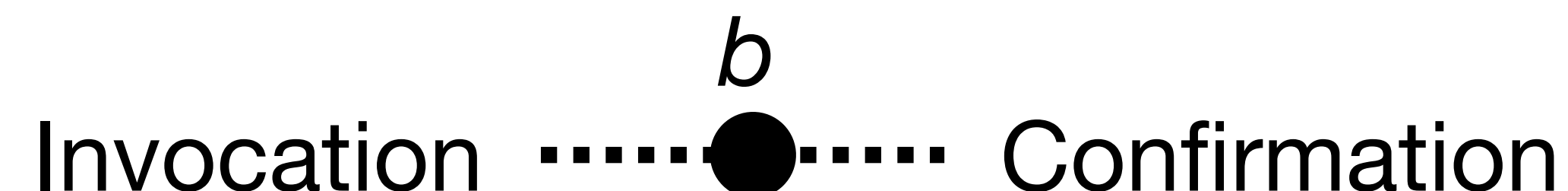
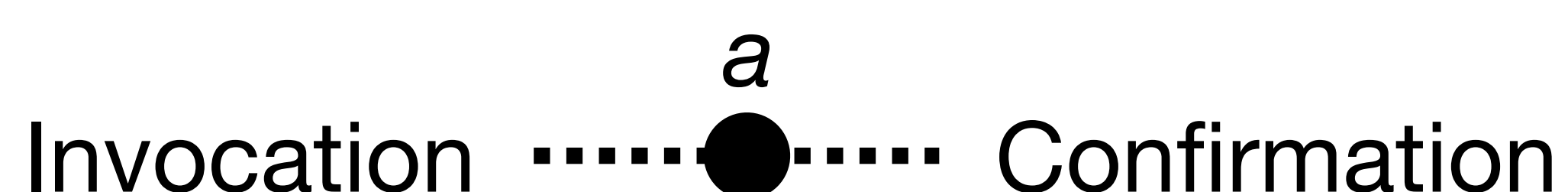
Execution must be
Linearizable

Linearizable Execution

A linearization of an execution is defined as a sequence of complete operations that appear atomically, **one after the other**.

Satisfying these conditions

1. Every read return the last value written
2. For any two operations a and b , if a precedes b in the **actual execution**, then a also appear before b in the linearization



Algorithm 4.10/11 Read-Impose Write-Consult-Majority

Extension of RIWM

Issue with RIWM:

- ***p*** makes a long sequence of writes
- ***q*** attempting to write but can't
- (because ***q*** is not in ***p***'s majority set, and its timestamp is lower)

Algorithm 4.10/11 Read-Impose Write-Consult-Majority

Timestamps must satisfy two properties

- Totally ordered

$$ts1 < ts2 < ts3$$

- Respect precedence

If ***a*** precedes ***b***: $ts(\mathbf{a}) < ts(\mathbf{b})$

Algorithm 4.10/11 Read-Impose Write-Consult-Majority

If each writer generates timestamps **independently**

A later write may appear **older**

Therefore timestamps must be **coordinated**

Algorithm 4.10/11 Read-Impose Write-Consult-Majority

Idea: **Distributed Timestamp Selection**

Before writing, a process **consults other processes**

Steps:

1. Collect timestamps from a **majority**
2. Select the **largest timestamp**
3. Generate a **new larger timestamp**

Algorithm 4.10: Read-Impose Write-Consult-Majority

Implements:

(N, N) -AtomicRegister, **instance** *nnar*.

Uses:

BestEffortBroadcast, **instance** *beb*;

PerfectPointToPointLinks, **instance** *pl*.

upon event $\langle nnar, Init \rangle$ **do**

$(ts, wr, val) := (0, 0, \perp)$;

acks := 0;

writeval := $\perp$;

rid := 0;

readlist := $[\perp]^N$;

readval := $\perp$;

reading := FALSE;

upon event $\langle nnar, Read \rangle$ **do**

rid := *rid* + 1;

acks := 0;

readlist := $[\perp]^N$;

reading := TRUE;

trigger $\langle beb, Broadcast \mid [READ, rid] \rangle$;

upon event $\langle beb, Deliver \mid p, [READ, r] \rangle$ **do**

trigger $\langle pl, Send \mid p, [VALUE, r, ts, wr, val] \rangle$;

upon event $\langle pl, Deliver \mid q, [VALUE, r, ts', wr', v'] \rangle$ **such that** $r = rid$ **do**

readlist[*q*] := (ts', wr', v') ;

if $\#(readlist) > N/2$ **then**

$(maxts, rr, readval) := highest(readlist)$;

readlist := $[\perp]^N$;

if *reading* = TRUE **then**

trigger $\langle beb, Broadcast \mid [WRITE, rid, maxts, rr, readval] \rangle$;

else

trigger $\langle beb, Broadcast \mid [WRITE, rid, maxts + 1, rank(self), writeval] \rangle$;

upon event $\langle nnar, Write \mid v \rangle$ **do**

rid := *rid* + 1;

writeval := *v*;

acks := 0;

readlist := $[\perp]^N$;

trigger $\langle beb, Broadcast \mid [READ, rid] \rangle$;

upon event $\langle beb, Deliver \mid p, [WRITE, r, ts', wr', v'] \rangle$ **do**

if (ts', wr') is larger than (ts, wr) **then**

$(ts, wr, val) := (ts', wr', v')$;

trigger $\langle pl, Send \mid p, [ACK, r] \rangle$;

upon event $\langle pl, Deliver \mid q, [ACK, r] \rangle$ **such that** $r = rid$ **do**

acks := *acks* + 1;

if *acks* > $N/2$ **then**

acks := 0;

if *reading* = TRUE **then**

reading := FALSE;

trigger $\langle nnar, ReadReturn \mid readval \rangle$;

else

trigger $\langle nnar, WriteReturn \rangle$;

Coding